Low-Latency Trading Engineering

A mastery-gated course plan for breaking into quantitative trading engineering

Primary path: Low-latency C++ / execution engineering

Secondary paths: Trading strategy | Infrastructure & platform | Performance | Acceleration

Course philosophy: *Build. Measure. Explain. Interview. Iterate.*

Prepared October 2026

Contents

Mission and target candidate 3

How the curriculum works 3

Curriculum map 4

Stage 0 - Baseline and engineering environment 5

Stage 1 - Modern C++ for systems programming 6

Stage 2 - Concurrency and the C++ memory model 7

Stage 3 - Computer architecture and performance engineering 9

Stage 4 - Linux systems and networking 10

Stage 5 - Low-latency engineering 12

Stage 6 - Electronic trading systems 12

Stage 7 - Flagship project: Mercury 13

Stage 8 - Specialization branches 15

Open-source contribution track 16

Interview training track 17

Application gates 18

Application strategy 19

Anti-rabbit-hole rules 20

Recommended resource stack 21

Asynchronous study loop 22

Portfolio structure 22

What exceptionally strong means 22

Definition of done 23

Selected references 23

> *This is not a calendar-based bootcamp. Advance when you can demonstrate the exit criteria. Begin applying before the curriculum is complete.*

Mission and target candidate

The primary goal is to become an exceptionally strong candidate for low-latency C++ / execution engineering roles at quantitative trading firms.

Secondary paths remain deliberately open:

- Trading strategy engineering.
- Trading infrastructure.
- Platform engineering.
- Performance engineering.
- Market-data engineering.
- Hardware/software acceleration.

The curriculum is asynchronous and mastery-gated. There are no arbitrary week counts. You advance when you can demonstrate the skill.

> *The course also contains explicit application gates so studying cannot expand indefinitely and prevent you from actually interviewing.*

The target candidate

At the end, your profile should read approximately like this:

> *Senior backend/distributed-systems engineer with strong C++ systems-programming skills, practical knowledge of Linux internals, networking, concurrency and computer architecture, and demonstrated ability to measure and optimize latency-sensitive systems. Built a realistic electronic-trading system in C++ and contributed to production-quality open-source systems software.*

That is much stronger than:

> *Backend engineer who studied some C++ and did LeetCode.*

You do not need to pretend you already have professional HFT experience. A credible story is:

> *I have spent my career building backend and distributed systems. I became increasingly interested in performance-sensitive systems, so I deliberately developed much deeper C++/Linux/performance expertise and built real systems demonstrating it.*

How the curriculum works

Every stage combines four forms of work:

Component	Purpose
Knowledge	Understand the underlying systems.
Implementation	Prove you can actually build them.
Measurement	Prove that you understand performance.
Interview practice	Convert knowledge into interview performance.

> *For every major concept you study, produce something executable, measurable, explainable, or interviewable.*

Reading about caches is not sufficient; demonstrate cache effects with a benchmark. Reading about atomics is not sufficient; implement and explain a concurrent queue. Reading about networking is not sufficient; write socket code. Reading about market making is not sufficient; run a strategy against your exchange simulator.

Curriculum map

Stage	Focus	Outcome
Stage 0	Baseline and environment	Establish tools and starting point
Stage 1	Modern C++ fluency	Language and resource-management competence
Stage 2	Concurrency and memory model	Reason correctly across CPU cores
Stage 3	Architecture and performance	Measure, profile and optimize
Stage 4	Linux and networking	Understand the machine and network stack
Stage 5	Low-latency engineering	Build for predictability and latency
Stage 6	Electronic trading systems	Learn the domain by implementing it
Stage 7	Mercury flagship project	Integrate the entire skill stack
Stage 8	Specialization	Execution, strategy, platform or acceleration

Throughout the entire program, three tracks continue in parallel:

- Interview practice and DSA.
- Open-source contribution work.
- Applications once the first gate is crossed.

The critical path ends before acceleration or FPGA specialization. You should be applying before you have mastered every advanced topic.

Stage 0 - Baseline and engineering environment

Objective: Establish your starting point and create the environment you will use for the rest of the curriculum. This is not a learn-programming phase.

Environment

Eventually you want access to bare-metal x86-64 Linux. Early exercises can run elsewhere, but serious latency measurements should not be performed primarily inside a noisy cloud VM.

Install and become comfortable with:

- GCC.
- Clang.
- CMake.
- Ninja.
- GDB.
- Git.
- AddressSanitizer.
- UndefinedBehaviorSanitizer.
- ThreadSanitizer.
- perf.
- strace.
- taskset.
- numactl.
- lscpu.
- Google Benchmark.

Baseline assessment

Without substantial preparation, attempt the following. Save the results; this is a baseline, not an exam.

C++

- Dynamic array.
- Hash map.
- Thread pool.
- TCP echo server.

Algorithms

- Graph traversal.
- Heap / top-K.
- Binary search.
- Sliding window.
- Interval problem.
- Basic dynamic programming.

Systems

- Explain process vs thread.
- Explain virtual memory.
- Explain context switching.
- Explain TCP vs UDP.
- Explain mutex vs atomic.
- Explain a cache line.
- Explain a race condition.

Performance

Take a simple C++ program and answer: Where is the program spending its CPU time? Use an actual profiling tool rather than guessing.

Stage 1 - Modern C++ for systems programming

Goal: Become comfortable enough with C++ that the language no longer consumes most of your mental bandwidth. The objective is production C++ fluency, not encyclopedic language-lawyer knowledge. Target C++20 as your working language and recognize useful C++23 features.

Core topics

Object lifetime

- Automatic and dynamic storage.
- RAII.
- Destructors.
- Ownership.
- References and pointers.
- Dangling references.
- Object lifetime.
- Initialization.

Value semantics

- Copy construction.
- Move construction.
- Copy assignment.
- Move assignment.
- Perfect forwarding conceptually.
- Rule of Zero.
- Rule of Five.

Standard library

- vector.
- array.
- span.
- string.
- string_view.
- unordered_map.
- map.
- optional.
- variant.
- unique_ptr.
- shared_ptr.
- Algorithms.
- Ranges at a practical level.

Templates

- Function templates.
- Class templates.
- Concepts.
- Type traits.
- Generic algorithms.

Do not disappear into template metaprogramming.

Error handling

- Exceptions.
- Error codes.
- optional.
- expected-style results.
- Assertions.

Memory

- Stack vs heap.
- Allocations.
- Alignment.
- Contiguous memory.
- Object layout.
- Padding.

Exercises: cpp-systems-lab

Build a repository called cpp-systems-lab. Each experiment should include an implementation, test, benchmark where relevant, short explanation, and observations.

Allocation experiment

- Allocate one million objects individually.
- Store them contiguously.
- Preallocate.
- Measure the differences.

String experiment

- Copies.
- Moves.
- string_view.

Container experiment

- vector.
- Linked structures.
- Hash maps.
- Different access patterns.

Virtual dispatch experiment

- Direct calls.
- Polymorphic calls.
- Measure rather than assume.

> *The habit matters more than the number: hypothesis -> measurement -> explanation.*

Stage 1 exit gate

- Implement normal programs without fighting C++.
- Explain RAII naturally.
- Identify ownership in unfamiliar code.
- Explain copy vs move.
- Debug lifetime bugs.
- Reason about allocations.
- Use STL containers appropriately.
- Write tests and benchmarks.
- Read moderately complicated C++ code.

> *Do not wait until you know every corner of C++. Proceed once the exit gate is solid.*

Stage 2 - Concurrency and the C++ memory model

Goal: Move from "I know threads" to "I can reason about communication between CPU cores." This is one of the most important stages in the program.

Ordinary concurrency first

- Threads.
- Mutexes.
- Condition variables.
- Semaphores.
- Producer/consumer.
- Thread pools.
- Deadlock.
- Starvation.
- Contention.

Implement conventional synchronization before reaching for atomics.

Atomics and memory ordering

- Atomic load/store.
- Compare-and-swap.
- fetch_add.
- Memory visibility.
- Happens-before.
- Synchronization relationships.
- relaxed.
- acquire.
- release.
- acquire-release.
- sequential consistency.

> *Do not memorize the standard. Learn to reason about concrete examples and explain why the ordering is sufficient.*

Concurrency project progression

Project A - Blocking queue

- Implement BlockingQueue<T>.
- Use mutex + condition variable.
- Test shutdown and edge cases heavily.

Project B - Thread pool

- Shutdown semantics.
- Task ownership.
- Task exceptions.
- Queue contention.
- Work distribution.

Project C - SPSC ring buffer

- Single producer / single consumer.
- Lock-free steady-state path.
- Compare against an established implementation afterward.

Benchmark at minimum:

- Mutex queue.
- Condition-variable queue.
- SPSC ring.
- Throughput.
- Median latency.
- p99 latency.

Then investigate:

- Padding.
- False sharing.
- Memory ordering.
- Busy spinning.

Stage 2 interview capability

- When would you use a mutex rather than atomics?
- What is false sharing?
- Why does this queue need acquire/release?
- What happens if producer and consumer counters share a cache line?
- Why can lock-free code still be slow?
- What does wait-free mean?

Stage 3 - Computer architecture and performance engineering

Core mental model

You need to understand the path: C++ source -> compiler -> assembly -> CPU pipeline -> cache hierarchy -> memory.

CPU architecture

- Instructions.
- Registers.
- Pipelines.
- Superscalar execution.
- Out-of-order execution.
- Branch prediction.

Memory hierarchy

- Registers.
- L1.
- L2.
- L3.
- DRAM.
- Why memory latency can dominate arithmetic.

Cache behavior

- Cache lines.
- Locality.
- False sharing.
- Cache misses.
- Prefetching.

Layout

- Array of Structures.
- Structure of Arrays.
- Choosing layout from access patterns.

SIMD and compiler behavior

- Vector instructions.
- Auto-vectorization.
- Basic intrinsics.
- Generated assembly.
- Optimization levels.
- Inlining.
- Constant propagation.
- Loop transformations.

Primary asynchronous course

Use selected portions of MIT 6.172, Performance Engineering of Software Systems, as a performance-engineering spine. Prioritize measurement, caching, storage allocation, parallelism, synchronization, and tuning. Do not feel obligated to complete every assignment sequentially.

Performance laboratory

- Cache locality: random traversal vs sequential traversal.
- Cache lines: demonstrate false sharing.
- Branch prediction: predictable vs unpredictable branches.
- Allocation: hot-loop heap allocation vs preallocation.
- Layout: AoS vs SoA.
- SIMD: scalar vs vectorized implementation.
- Compiler optimization: inspect generated assembly.

Measurement discipline

Latency numbers are easy to lie with accidentally. Learn to control or at least understand:

- Warmup.
- CPU frequency.
- Scheduler interference.
- CPU affinity.
- Background tasks.
- Compiler optimization.
- Dead-code elimination.
- Clock sources.
- Sample distributions.

Stop reporting only averages. Prefer median, p95, p99, p99.9, and maximum when useful.

Stage 3 exit gate

1. Measure a slow program.
2. Profile it.
3. Form a hypothesis.
4. Alter one thing.
5. Benchmark again.
6. Explain why performance changed.

> *That workflow is more important than memorizing fast-C++ tricks.*

Stage 4 - Linux systems and networking

Goal: Become comfortable reasoning about the machine beneath your process.

Linux

- Processes.
- Threads.
- Scheduling.
- Syscalls.
- File descriptors.
- Signals.
- Virtual memory.
- Page faults.
- Memory mapping.
- Shared memory.
- CPU affinity.
- NUMA.
- Huge pages.
- Interrupt behavior.
- Context switches.

Regularly use:

```
perf stat
perf record
perf report
strace
taskset
numactl
```

Networking

Start from sockets and build rather than only reading.

- TCP server.
- UDP server.
- Binary protocol.
- Multicast receiver.

Learn:

- Sockets.
- TCP.
- UDP.
- Multicast.
- Packet loss.
- Sequence numbers.
- Buffering.
- Kernel networking.
- Serialization.

Understand the tradeoff: TCP provides reliability and ordering; UDP minimizes protocol overhead and gives the application more control over recovery.

Networking project - toy market-data feed

Publisher emits:

- Instrument ID.
- Sequence number.
- Bid.
- Ask.
- Quantity.
- Timestamp.

Subscriber:

- Receives packets.
- Parses them.
- Detects sequence gaps.
- Updates market state.
- Measures end-to-end latency.

This becomes part of the eventual trading-system project.

Stage 5 - Low-latency engineering

The question now changes from "How do I make C++ fast?" to "How do I make the entire system predictable?"

Allocation avoidance

- Avoid allocations on the critical path.
- Preallocation.
- Object pools when justified.
- Fixed-capacity containers where appropriate.

Data-oriented design

Organize memory based on how it is accessed, not only object-oriented abstractions.

CPU pinning and busy spinning

- Understand why critical threads may be pinned to cores.
- Understand why a system may intentionally consume a CPU core instead of blocking.
- Measure the latency/CPU tradeoff.

Lock avoidance and queue design

- Single-writer designs.
- SPSC.
- MPSC.
- MPMC.
- Why lock-free does not automatically mean faster.

Batching, syscalls and jitter

- Latency/throughput tradeoff.
- Why entering the kernel matters.
- Tail latency and variance.
- Sources of scheduling jitter.

Kernel bypass

Study DPDK only after normal networking. Understand the architecture and why poll-mode drivers can bypass the traditional kernel networking path. Eventually build a basic DPDK application if the hardware and environment permit.

> *Do not make DPDK expertise a prerequisite for applying. Understand why it exists before trying to master it.*

io_uring

Learn the model and experiment with liburing, while remembering that asynchronous I/O and ultra-low-latency trading solve related but distinct problems.

Stage 6 - Electronic trading systems

Only now introduce significant trading-specific material. You do not need a finance degree; you need to understand the environment your software operates in.

Market microstructure

- Exchanges.
- Securities.
- Futures.
- Options at a conceptual level.
- Bids and asks.
- Spread.
- Liquidity.
- Makers and takers.
- Market orders.
- Limit orders.
- Order books.
- Price/time priority.
- Fills.
- Cancellations.
- Partial fills.

Then understand:

- Market data.
- Order entry.
- Execution acknowledgements.
- Drop copy.
- Risk checks.
- Position.
- P&L.

Order book implementation

Implement one. Represent levels and process add, modify, cancel, and trade messages. Start simple and correct, then benchmark different structures.

> *This is a perfect place to demonstrate that the theoretically elegant data structure is not always the fastest structure for the real access pattern.*

Exchange simulator

Build a small exchange that accepts new orders and cancellations, generates acknowledgements and fills, publishes market-data updates, and supports price/time priority.

FIX

Understand FIX conceptually and study a mature implementation such as QuickFIX. If useful, build an external gateway around QuickFIX, but do not make FIX the hot-path protocol in your simulated HFT stack. Compare it conceptually against compact binary exchange protocols.

Stage 7 - Flagship project: Mercury

This is the most important differentiator in the curriculum. Rather than ten toy repositories, build one unusually good system.

Mercury

> *A simulated low-latency electronic trading platform that looks like something an experienced engineer built, not a bootcamp portfolio project.*

Conceptual architecture

```
Market-data generator / exchange -> network receiver -> market-data parser -> order book
-> strategy -> risk -> order gateway -> exchange simulator
```

Cross-cutting capabilities: telemetry, deterministic replay, tests, and benchmark harness.

Milestone 1 - Exchange

- Order entry.
- Cancellations.
- Price/time matching.
- Fills.
- Correctness tests first.

Milestone 2 - Market data

- Book updates.
- Sequence numbers.
- Trades.
- Binary protocol.

Milestone 3 - Client market-data handler

- Receive and parse updates.
- Detect gaps.
- Maintain a local order book.

Milestone 4 - Strategy

- Toy market maker, spread strategy, or imbalance signal.
- Use the strategy to exercise the system.
- Do not make unrealistic profitability claims.

Milestone 5 - Execution

- Strategy generates orders.
- Risk validates them.
- Gateway sends them.
- Receive fills.
- Track position.

Milestone 6 - Multithreading

- Separate network, strategy, and execution.
- Use queues between components.
- Reuse your SPSC queue where appropriate.

Milestone 7 - Hot-path constraints

- No dynamic allocations during steady-state processing.
- Predictable memory layout.
- Minimal copies.
- Bounded queues.
- CPU affinity where appropriate.
- Document each design choice.

Milestone 8 - Benchmarking

- Market-data pipeline: packet -> parse -> book update -> strategy decision.
- Execution pipeline: strategy decision -> risk -> order transmission.
- Report p50, p95, p99, and p99.9.

Milestone 9 - Profiling

- Use perf.
- Publish hotspot analysis.
- Examine cache behavior.
- Examine branch behavior.
- Inspect context switches where relevant.
- Show before/after optimization results and explanation.

Milestone 10 - Replay

- Record market events.
- Allow deterministic replay.
- Use replay for debugging, testing, simulation, and strategy research.

The README matters

The repository should be able to support a 30-minute technical discussion. Document:

- Problem: what system did you build?
- Architecture: include a diagram.
- Design decisions: why threads are partitioned this way.
- Performance methodology: how measurements were performed.
- Results: distributions and benchmarks.
- Profiling discoveries: what actually mattered.
- Tradeoffs: what did you intentionally not optimize?
- Future work: show awareness without pretending everything is complete.

Stage 8 - Specialization branches

Low-latency execution remains the default branch. Choose additional depth based on real job targets rather than curiosity alone.

Branch A - Low-latency execution

- C++ object layout.
- Allocators.
- Lock-free structures.
- Memory ordering.
- Linux scheduling.
- NUMA.
- CPU isolation.
- NIC queues.
- Multicast.
- Packet processing.
- Kernel bypass.
- Timestamping.
- PTP concepts.
- Exchange binary protocols.

Study real high-performance messaging systems such as Aeron and compare their design ideas against your own transport.

Branch B - Trading strategy engineering

- Market microstructure.
- Event-driven simulation.
- Backtesting.
- Transaction costs.
- Slippage.
- Queue position.
- Strategy evaluation.

Enhance Mercury with historical or synthetic replay, a strategy plug-in interface, and performance analytics. The engineering challenge matters more than inventing alpha.

Branch C - Infrastructure / platform

- Distributed systems.
- Market-data storage.
- Research computation.
- Data pipelines.
- Experiment reproducibility.
- Scheduling.
- Observability.
- Fault tolerance.
- Deployment.

Enhance Mercury with a persistent event log, distributed replay workers, strategy experiment runner, metrics, tracing, and failure recovery. This path maps especially well to existing backend/distributed-systems experience.

Branch D - Acceleration

Do this after Stage 5, not before.

Level 1 - SIMD

- Compiler auto-vectorization.
- SIMD intrinsics.
- Vector-friendly data layout.

Level 2 - Networking acceleration

- RSS.
- Kernel bypass.
- DPDK.
- NIC queues.
- Polling.

Level 3 - FPGA concepts

- RTL.
- Pipelines.
- Clock cycles.
- Fixed-width arithmetic.
- Verilog/SystemVerilog basics.
- Implement something narrow such as a binary market-data parser or simple risk check.

FPGA competence is a later differentiator, not a prerequisite for applying to C++ roles.

Open-source contribution track

The goal is not to collect GitHub activity. The goal is to have another experienced engineer accept technical work you contributed to a real systems project.

Contribution ladder

1. Build the project from source.
2. Run its tests.
3. Understand one subsystem.
4. Find a reproducible problem.
5. Add a test or benchmark.
6. Submit a technically meaningful fix.
7. Respond well to review.

Good candidate projects

Project	Why it fits	Difficulty
QuickFIX	Trading-domain C++ code; parsing, sessions, tests, performance	Moderate
Aeron	High-performance messaging and transport	Advanced
Google Benchmark	Directly supports performance engineering	Moderate
Boost.Asio	Networking credibility	Advanced
liburing	Linux async-I/O credibility	Advanced
DPDK	Extremely relevant to packet-path performance	Very advanced

What counts as impressive?

- Fixed a concurrency bug and added a regression test.
- Added benchmark coverage and improved parser latency.
- Diagnosed a Linux-specific performance regression and submitted a fix.

Documentation work is legitimate, but it does not create the same technical signal for this particular goal.

Interview training track

Begin early and increase intensity over time. Do not wait for the course to be finished.

Phase I - During Stages 1-2

- Basic DSA.
- C++ coding.
- Explaining code clearly.

Phase II - During Stages 3-4

- Operating systems.
- Networking.
- Concurrency.
- C++ internals.
- Performance.

Phase III - During Stages 5-7

- System design.
- Low-latency design.
- Market-data systems.
- Trading-system design.
- Project deep dives.

Algorithm expectations

- Arrays.
- Strings.
- Hashing.
- Stacks.
- Queues.
- Heaps.
- Trees.
- Graphs.
- Binary search.
- Two pointers.
- Sliding windows.
- BFS.
- DFS.
- Basic dynamic programming.

> *Do not optimize for solving 500 problems. A stronger target is: given an unfamiliar medium problem, reason toward a correct solution, implement it cleanly in C++, test it, and discuss complexity.*

Asynchronous mock interviews

Set up a 45-60 minute session, use an unseen problem, record yourself, and speak continuously. Review:

- Did I clarify the problem?
- Did I formulate an approach before coding?
- Did I communicate complexity?
- Did I test edge cases?
- Did I get stuck silently?
- Was my C++ clean?

Do the same for system design. Explain a prompt such as "Design a market-data distribution system" to a camera, then critique yourself. External mock interviews are useful occasionally but the curriculum does not depend on synchronous participation.

Application gates

First application gate

Start applying once all of the following are true:

Area	Minimum threshold
C++	Comfortable writing normal C++20 without constant reference lookup
DSA	Most common medium patterns are familiar
Concurrency	Can implement and explain an SPSC queue plus ordinary synchronization
Operating systems	Comfortable with processes, threads, virtual memory and scheduling
Networking	Have written TCP and UDP socket programs
Performance	Can profile with perf and interpret results
Architecture	Understand cache/locality/allocation/false sharing
Trading	Understand order books and basic market structure
Portfolio	Mercury has a working exchange, market data and basic execution
Interviewing	Can complete realistic 45-minute practice interviews

> *At this point: START APPLYING. Mercury does not need to be finished. You do not need deep DPDK knowledge. You do not need FPGA. There will still be difficult LeetCode problems you cannot solve.*

Strong-candidate gate

Before heavily targeting the firms and teams you most want, aim for:

C++

- Lifetime.
- Allocation.
- Move semantics.
- STL behavior.
- Memory layout.
- Templates.
- Atomics.
- Concurrency.

Performance

- Explain at least three real optimizations you personally measured.

Linux

- Diagnose performance using system tools.

Networking

- Have implemented real socket systems.

Trading

- Explain exchange packet -> market data -> book -> strategy -> risk -> order -> execution.

Portfolio

- Realistic architecture.
- Tests.
- Benchmarks.
- Latency distributions.
- Profiling.
- Clear documentation.

Open source

- Preferably at least one merged technical contribution.

Interviews

- Repeatedly perform well in coding, systems, C++, performance and design under interview conditions.

Application strategy

Do not divide employers into qualified and unqualified. Apply across several role types.

Search titles such as:

- Low-Latency Software Engineer.
- C++ Software Engineer.
- Trading Systems Engineer.
- Execution Engineer.
- Market Data Engineer.
- Trading Platform Engineer.
- Trading Strategy Engineer.
- Quantitative Developer.
- Research Engineer.
- Performance Engineer.
- Infrastructure Engineer.

Application priority

Tier	Target	Reason
A	Low-latency C++ / execution	Direct aspiration
B	Trading strategy / market-data engineering	Highly desirable adjacent path
C	Trading infrastructure / platform	Excellent entry path using existing strengths

Getting into a trading firm as a strong platform engineer does not trap you permanently outside execution. It gets you inside the domain while you continue deepening C++ and performance skills.

Anti-rabbit-hole rules

1. **Do not learn advanced material merely because it exists.** Learn it when your project or target job demands it.
2. **Never read three books before building something.** Study enough to build. Build. Then return to theory.
3. **Measurement before optimization.** Never optimize Mercury because HFT systems probably do this. Profile it.
4. **Correct before lock-free.** A mutex-based implementation is often the correct first implementation.
5. **Do not postpone applications for FPGA.** FPGA is optional specialization.
6. **Do not postpone applications for quantitative finance.** You are applying as an engineer.
7. **Do not chase obscure C++ trivia.** Deep fundamentals beat trivia.
8. **Do not solve LeetCode forever.** Interview problems are a means to an end.
9. **Every deep dive must justify itself.** Will this improve interview performance, the project, an open-source contribution, or ability to do the job? If not, defer it.

Topics deliberately excluded from the critical path

- Stochastic calculus.
- Advanced derivatives pricing.
- Quantitative-research mathematics.
- CUDA.
- FPGA development.
- Compiler construction.
- Linux kernel development.
- Advanced template metaprogramming.
- Exotic lock-free algorithms.
- Formal verification.
- Writing custom allocators from scratch.
- Advanced networking hardware internals.

These can all become useful later. None should block applications.

Recommended resource stack

Keep the stack deliberately small. Avoid turning resource collection into study.

C++

- C++ Core Guidelines.
- cppreference.
- One practical modern C++ book - one is enough.

Concurrency

- C++ Concurrency in Action.
- cppreference atomics and memory model.
- Established queue implementations such as Folly for comparison.

Systems

- Computer Systems: A Programmer’s Perspective.
- Linux man pages.

Performance

- MIT 6.172.
- Systems Performance by Brendan Gregg.
- Google Benchmark.
- Linux perf.

Networking

- Beej’s Guide to Network Programming.
- Linux socket documentation.
- Your own socket implementations.

Trading

- Trading and Exchanges by Larry Harris.
- One serious microstructure text is enough initially.

Advanced systems

- Aeron.
- QuickFIX.
- DPDK.
- liburing.

Asynchronous study loop

Use a repeatable learning cycle that produces retrieval, transfer and evidence of competence:

Step	Action
Discover	Study the concept.
Retrieve	Close the resource and explain it yourself.
Implement	Write something that exercises it.
Break	Test edge cases and failure modes.
Measure	Benchmark it where relevant.
Explain	Write a short engineering note.
Revisit	Return later without notes and reconstruct the idea.

This supports asynchronous learning without relying on deadlines or synchronous lectures, while preventing passive familiarity from masquerading as mastery.

Portfolio structure

Do not build dozens of repositories. Aim for approximately:

Artifact	Purpose
cpp-systems-lab	Small performance, concurrency, memory and networking experiments.
mercury	Flagship low-latency trading-system project.
Open-source profile	A small number of real contributions to external systems projects.

> *Quality and technical depth matter dramatically more than repository count.*

What exceptionally strong means

You will not differentiate yourself merely because you know C++ syntax.

> *The differentiator is: I can explain what the hardware is doing, write the C++ that interacts with it, measure the resulting system, identify the bottleneck, improve it, and defend the tradeoffs.*

Combined with existing backend/distributed-systems experience, this creates a valuable hybrid:

```
distributed-systems maturity + low-level systems competence + performance engineering +
trading-domain experience from substantial personal work
```

That is a much stronger positioning than attempting to compete with new graduates solely on algorithm puzzles.

Definition of done

There is intentionally no final point where you "know low latency." The course succeeds when:

1. You have crossed the application gate.
2. You are actively interviewing.
3. Interview failures become input for further study.
4. You continue improving Mercury.
5. You make meaningful open-source contributions.
6. Your study becomes driven increasingly by real interview feedback.

At that point the loop changes from:

```
study -> study -> study -> maybe apply
```

to:

```
study -> build -> apply -> interview -> identify weakness -> study -> improve -> interview again
```

That is the intended steady state.

Selected references

C++ Core Guidelines:
https://github.com/isocpp/CppCoreGuidelines

cppreference: std::memory_order:
https://en.cppreference.com/w/cpp/atomic/memory_order

MIT 6.172 - Performance Engineering of Software Systems:
https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/

Beej's Guide to Network Programming:
https://beej.us/guide/bgnet/

Google Benchmark:
https://github.com/google/benchmark

Folly ProducerConsumerQueue:
https://github.com/facebook/folly/blob/main/folly/docs/ProducerConsumerQueue.md

DPDK Programmer's Guide:
https://doc.dpdk.org/guides/prog_guide/

liburing:
https://github.com/axboe/liburing

QuickFIX:
https://github.com/quickfix/quickfix

Aeron C++ Programming Guide:
https://github.com/aeron-io/aeron/wiki/Cpp-Programming-Guide

Boost.Asio:
https://github.com/boostorg/asio

Reference note

The resources above are intentionally selective. The course is designed to prevent resource sprawl: use the smallest set that supports the current mastery gate, project milestone, interview weakness, or open-source contribution.